

Machine Learning-Based Construction Cost Estimation

1.1 Introduction

Construction cost estimation is a critical decision-support activity in the early stages of project planning and budgeting. At the conceptual or Class 5 level, owners and contractors must make go/no-go decisions, compare alternatives, and allocate capital long before detailed quantities and drawings are available. AACE International’s cost estimate classification system describes Class 5 estimates as having very low scope definition and a typical accuracy range of approximately $\pm 20\%$ to $\pm 50\%$, depending on the sector and risk profile [1]. In practice, many organizations are satisfied when early estimates fall within roughly 25% of actual costs on average.

Traditional Class 5 estimates for civil and site work projects often rely on high-level historical rules of thumb, square-foot factors, or simple parametric relationships calibrated on small internal datasets. While these methods can be fast, they may not fully exploit the rich information now available in detailed cost histories, area cost factors, and contextual project attributes. Over the past two decades, the construction management literature has shown that data-driven and machine learning (ML) approaches can improve accuracy and consistency in early cost prediction [2–5]. Neural networks, case-based reasoning, support vector machines, and ensemble models such as random forests and gradient boosting have all been applied to various building and infrastructure datasets with promising results [6–11].

This project builds on that body of work in an industry setting. The goal is to retrain and update an ML-based Class 5 estimator for Construction Check, Inc.

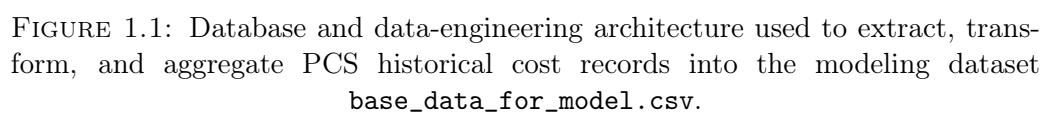
using a cleaned, CPI-normalized subset of their PCS historical project database, as described in the internal handoff brief [12]. The specific objective is to train a model that predicts total project cost (normalized to 2025 dollars) for a focused subset of civil and sitework project types, while meeting or improving upon a target Mean Absolute Percentage Error (MAPE) of 25% on unseen projects. In parallel, the project delivers a prototype Flask-based web application that exposes the trained model through a simple user interface for estimators and project managers.

1.2 Data Pipeline and Feature Engineering

1.2.1 Database and Pipeline Architecture

The historical cost data for this study comes from PCS internal tables that store projects, phases, line items, CSI codes, area cost factors, and related attributes. The production data pipeline was designed to mirror how an eventual application will access the same information. Figure 1.1 shows the high-level architecture, where a Python application layer connects securely to an Azure-hosted PostgreSQL instance via SQLAlchemy. The pipeline is implemented as a sequence of modular steps that can be run in full extraction mode or in a faster, “CSV-only” mode using previously exported data, controlled by an `All_Records` flag.

The first stage performs database diagnostics and data extraction. A comprehensive SQL query joins multiple PCS tables to create a project-level dataset that aggregates line-item costs, phases, project types, and area cost factors. This query computes total costs from detailed line items so that the resulting project-level totals are consistent with how PCS currently prices jobs. The full SQL output contains



7

1.2.2 Project Selection and Geographic Enrichment

Because the initial dataset spans a wide variety of building and infrastructure types, a subset of project types was selected based on teaching assistant (TA) guidance and Construction Check priorities. Four specific project types were retained for this study:

- Sidewalks, Curbs & Gutters
- Pavement Markers
- Parks & Landscaping
- Site Work

In the code, these filters reduce the working dataset from 167,359 to 18,917 relevant projects [13]. To enable geographic and regional analysis, the pipeline then enriches each project with county information derived from latitude and longitude coordinates. The US Census Geocoding API is used to map each project location to a county, and the results are merged back into the main dataset. The pipeline achieves a 100% county match rate for the selected projects, which in turn allows grouping and analysis by state, county, and later by data-driven regional clusters.

In addition to county, each project is labeled as “Urban” or “Rural” using a curated list of urban cities by state. The majority of projects in this subset are classified as rural: 16,973 of the 18,917 records fall into the rural category, with the remainder labeled as urban or unknown [13]. This rural dominance highlights the type of infrastructure that PCS commonly delivers in this sample.

1.2.3 Cost Normalization and Inflation Adjustment

Because projects in the historical dataset span multiple years, raw costs are not directly comparable. To address this, all costs are normalized to constant 2025 dollars using official Consumer Price Index (CPI) data from the U.S. Bureau of Labor Statistics [14]. For each project year t , an inflation factor α_t is constructed from the CPI:

$$\alpha_t = \frac{\text{CPI}_{2025}}{\text{CPI}_t},$$

and the normalized total cost is

$$C_{\text{norm},i} = \alpha_{t(i)} \times C_{\text{nominal},i},$$

where $C_{\text{nominal},i}$ is the project's original total cost and $t(i)$ is the project's completion year. This normalization is applied consistently to all relevant cost columns. Across the dataset, the average inflation adjustment is approximately 9.3%, so some projects see relatively modest changes while older projects experience more substantial upward adjustments [13].

During the normalization and cleaning phase, missing values in key numerical fields are handled using median imputation, which avoids distorting the distribution tails and preserves the robustness of downstream models. At this point, the dataset is ready for outlier detection and exploratory data analysis.

1.2.4 Complexity, Budget Range, and Derived Features

To provide richer structure for analysis and modeling, several derived features are engineered from the raw PCS data:

- **Official budget range:** projects are mapped into discrete budget brackets such as “Less than \$1M” or “\$1M–\$2M” based on PCS-defined ranges. This field is used both descriptively and as an input to the model, capturing how PCS originally framed the estimate.
- **CIQS-style complexity categories:** division counts and unique CSI group counts are combined with project type information to derive seven ordinal complexity categories, labeled “Category 1” (simplest) through “Category 7” (most complex). A separate numeric variable `complexity_numeric` is extracted from these labels for correlation analysis and modeling.
- **Area cost factor (ACF):** area cost factors are carried through the pipeline and later used to investigate how regional cost multipliers relate to normalized costs.

Altogether, these steps culminate in a cleaned dataset `df_clean` with 17,025 records after outlier removal, which is exported to `base_data_for_model.csv` for modeling [15].

1.3 Exploratory Data Analysis

1.3.1 Outlier Removal and Overall Cost Distribution

Before modeling, extremely low and high projects are trimmed to reduce the influence of outliers. Starting from 18,917 filtered projects, the bottom 5% and top 5% of normalized total costs are removed. This corresponds to costs below \$105,265 and above \$5,760,218, resulting in a clean modeling dataset of 17,025 projects and the removal of 1,892 outlier records [15]. After outlier removal, the distribution of total normalized cost has a mean of approximately \$1,142,250, a median of \$689,021, and a cost range from \$105,275 to about \$5,758,659. The coefficient of variation is about 101.5%, reflecting substantial heterogeneity in project scale and scope [15].

Figure 1.2 shows the histogram of normalized total project cost, while Figure 1.3 presents the same distribution on a log scale. The raw distribution is right-skewed, with a long tail of high-cost projects. Taking the logarithm of cost produces a much more symmetric distribution, which provides intuition for why tree-based models and log-transformed linear models often perform well on such data.

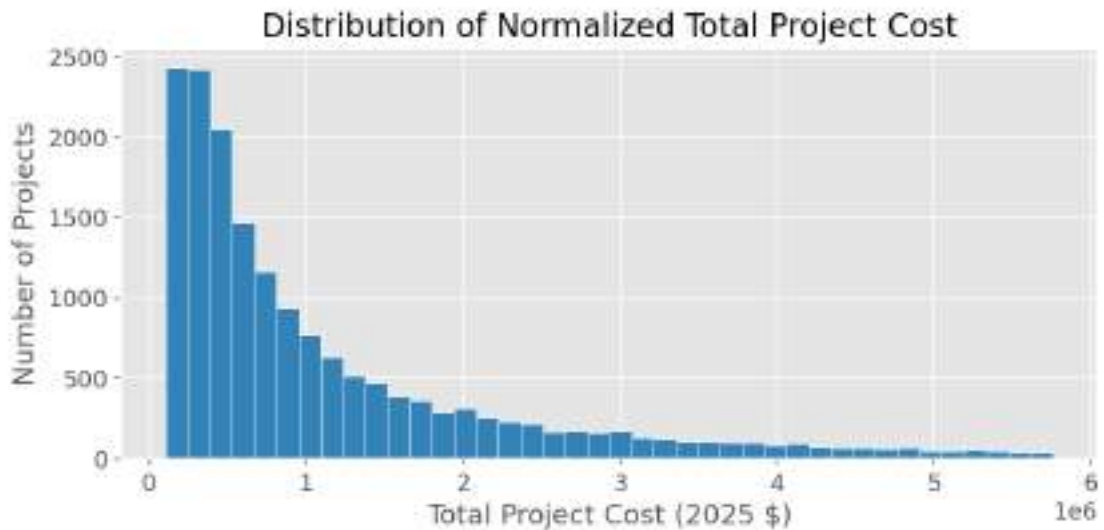


FIGURE 1.2: Distribution of normalized total project cost (2025 dollars) after removing the bottom and top 5% of projects by cost (17,025 projects).



FIGURE 1.3: Log-transformed distribution of normalized total project cost. The log scale reveals a more symmetric structure, which is helpful for understanding model assumptions and variance.

1.3.2 Project Mix and Cost by Category

Within the cleaned dataset, projects fall into three broader categories: Water & Sewer, Civil, and Site Work & Landscaping. These categories aggregate the four TA-specified project types and reflect how PCS organizes its internal cost data. The category-level summary from the EDA notebook shows that Water & Sewer has 6,147 projects, Civil has 4,922 projects, and Site Work & Landscaping has 5,956 projects, for a total of 17,025 projects [15]. Mean normalized costs are highest for Water & Sewer at approximately \$1,310,815, followed by Civil at \$1,145,620, and lowest for Site Work & Landscaping at about \$965,493. The ratio of the most expensive to least expensive category is roughly 1.4:1, indicating meaningful cost differentiation by category.

Figure 1.4 summarizes the frequency distribution of project types, while Figure 1.5 compares median or mean costs by type. Together, these plots confirm that the modeling dataset is dominated by mid-sized civil and sitework projects with substantial

but not extreme cost variability.

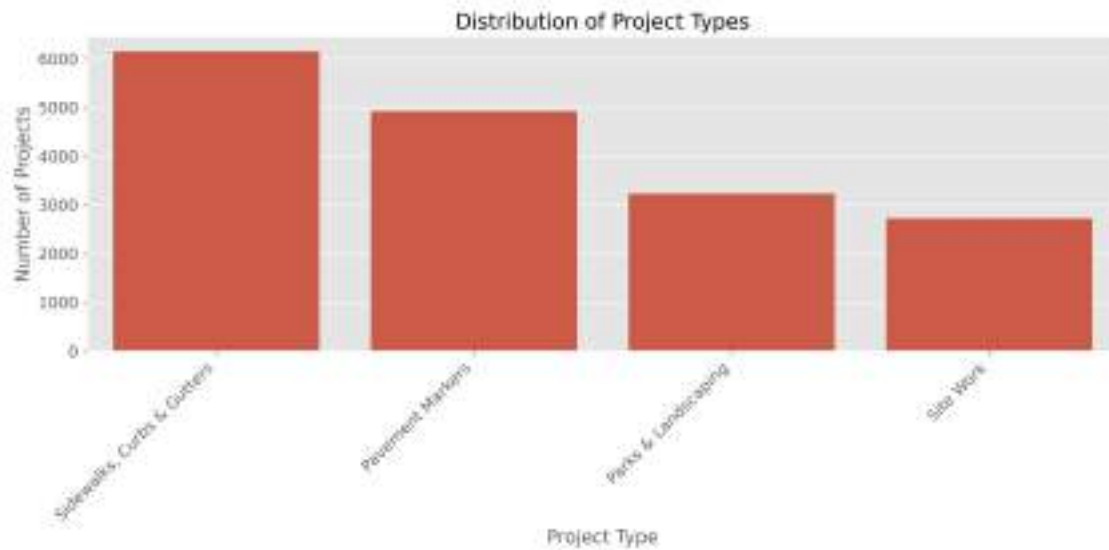


FIGURE 1.4: Distribution of project types in the cleaned dataset. The four retained project types (Sidewalks, Curbs & Gutters; Pavement Markers; Parks & Landscaping; Site Work) are grouped into three higher-level categories for modeling.

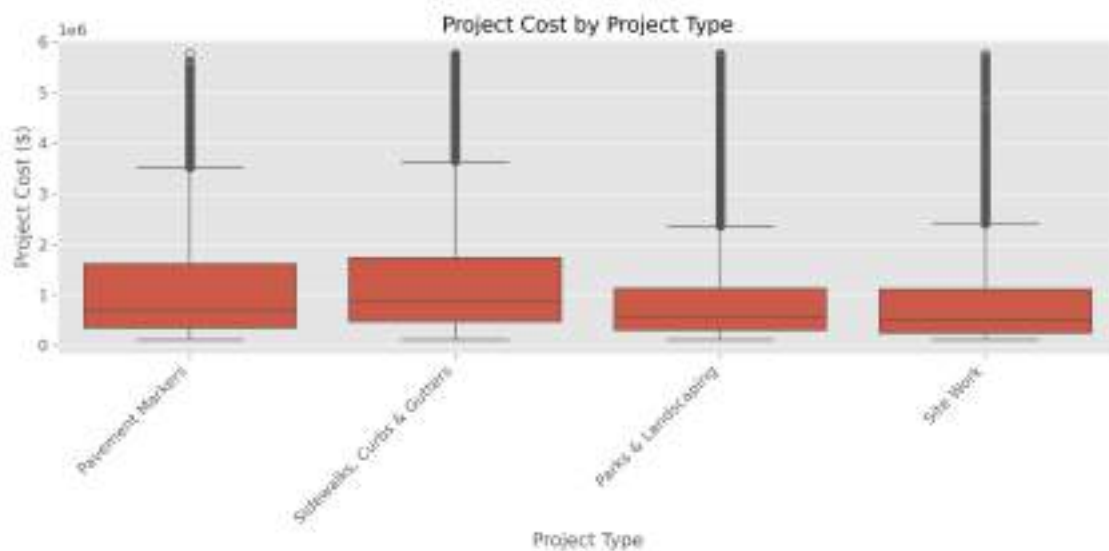


FIGURE 1.5: Normalized project cost by type or category, highlighting that Water & Sewer projects are, on average, more expensive than Civil and Site Work & Landscaping projects.

1.3.3 Complexity and Area Cost Factors

A key theme in the EDA is understanding how engineered features such as complexity and area cost factors relate to normalized costs. The CIQS-style complexity categories reveal a clear monotonic pattern: projects in Category 1 (simplest) have mean normalized costs around \$710,864, while Category 7 projects (most complex) average over \$2,300,000. The number of CSI divisions and line items also grows with complexity category, confirming that the derived complexity score captures real differences in scope [15].

The numeric complexity score `complexity_numeric` is moderately correlated with normalized cost, with a Pearson correlation of approximately 0.379 [13]. Figure 1.6 illustrates this relationship. While complexity alone does not fully explain cost, it provides a meaningful gradient that, when combined with other features, improves model performance.

Area cost factors (ACFs) provide another lens on regional variation. The EDA reports a modest positive correlation of around 0.170 between ACF and normalized project cost [15]. States with higher ACFs, such as California or Massachusetts, tend to exhibit higher mean normalized costs, while states with lower ACFs have comparatively lower average costs. This supports the inclusion of ACF in the feature set, even if it is not the dominant driver.

1.3.4 Regional Clusters and Categorical Encoding

Because county and state are high-cardinality categorical variables, the modeling pipeline introduces a compact “region” feature derived from unsupervised clustering of locations and cost patterns. Projects are assigned to four regions, labeled Region 0

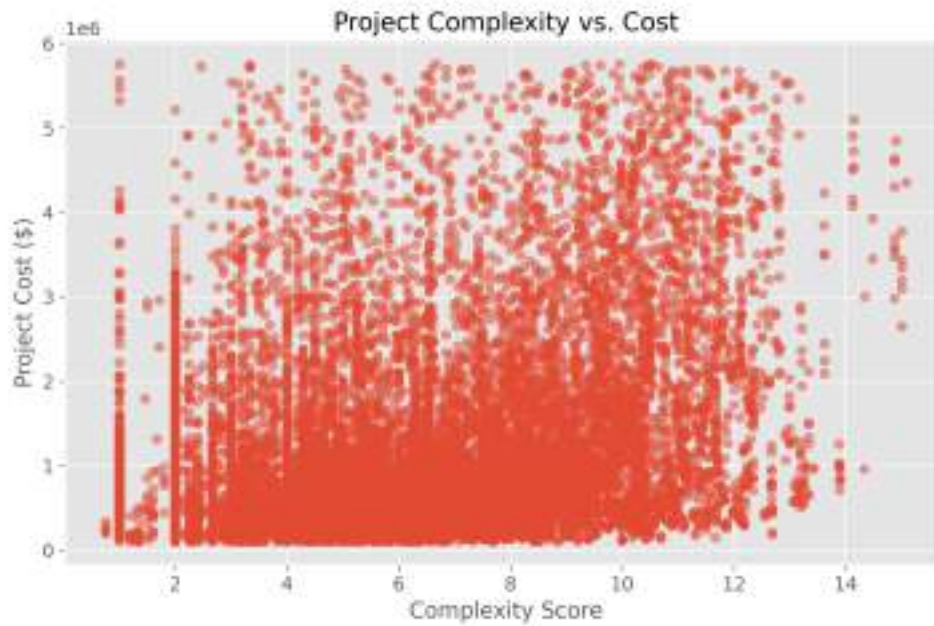


FIGURE 1.6: Relationship between CIQS-style complexity categories and normalized total cost. Higher complexity categories are associated with larger and more expensive projects.

through Region 3, which capture broad geographic and cost similarities. Figure 1.7 shows the resulting clusters, with each point representing a project labeled by its assigned region. These regions are ultimately encoded as one-hot categorical variables alongside other features.

1.4 Modeling Approach

1.4.1 Feature Set and Preprocessing

The modeling dataset `base_data_for_model.csv` contains 17,025 projects with a mix of numerical and categorical features. The target variable is

$$y = \text{total_project_cost_normalized_2025},$$

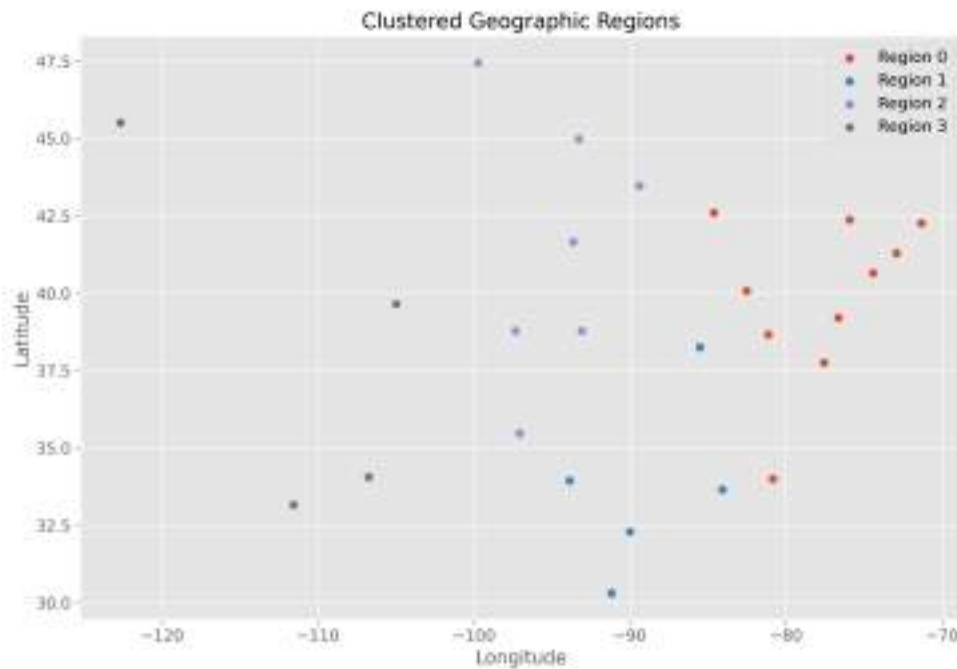


FIGURE 1.7: Data-driven regional clusters used as a compact geographic feature. Projects are grouped into four regions based on location and cost characteristics.

the CPI-adjusted total project cost. Numerical predictors include project square footage (where available), ACF, complexity score, counts of CSI divisions and unique CSI groups, and derived measures such as budget midpoints. Categorical predictors include project type, project category, construction category, area type (urban/rural), project state, county, region, CIQS complexity category, and official budget range.

The models are trained using a scikit-learn pipeline that combines preprocessing with the estimator. Numerical features are passed through a `StandardScaler`, while categorical features are transformed using a one-hot encoder with `handle_unknown='ignore'` to ensure robustness to new categories at inference time. This combined preprocessor is then composed with the regression model using `Pipeline`, ensuring that cross-validation and test evaluation apply consistent transformations.

The data is split into training and test sets using an 80/20 split. The training set contains 13,620 projects and the held-out test set contains 3,405 projects, stratified only implicitly through random shuffling. The same random state is used across models to ensure fair comparison.

1.4.2 Models and Evaluation Metrics

Four ensemble models are trained and evaluated:

- Random Forest Regressor
- Gradient Boosting Regressor
- XGBoost Regressor
- LightGBM Regressor

Each model is trained with 100 trees (or boosting iterations) and default depth-related parameters, providing a strong baseline without extensive hyperparameter tuning. For each model, five-fold cross-validation is performed on the training set using negative MAPE as the scoring function.

Three standard regression metrics are used:

1.4.3 Mean Absolute Percentage Error (MAPE).

For n projects with true costs y_i and predictions $\hat{y}_i$, the MAPE is

$$\text{MAPE} = \frac{100}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right|.$$

This metric expresses average absolute error as a percentage of the true value and directly aligns with Class 5 accuracy targets.

1.4.4 Root Mean Squared Error (RMSE).

The RMSE in dollars is given by

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2},$$

which penalizes larger errors more heavily and is useful for assessing the absolute scale of deviations on large projects.

1.4.5 Coefficient of Determination (R^2)

The R^2 score measures the proportion of variance in costs explained by the model:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2},$$

where $\bar{y}$ is the mean of the observed costs. A value close to 1 indicates that the model captures most of the variability in project costs.

For each model, the notebook records cross-validated MAPE on the training set (CV MAPE), in-sample training MAPE, test MAPE on the held-out 20%, test RMSE in dollars, and test R^2 .

1.5 Results

1.5.1 Model Comparison

Table 1.1 summarizes the performance of all four models evaluated in the organized notebook. Random Forest achieves the best overall balance of accuracy and generalization, with a test MAPE of approximately 22.18%, outperforming the 25% target by about 2.82 percentage points. It also attains the highest test R^2 and the lowest RMSE among the four models.

Model	CV MAPE (%)	Train MAPE (%)	Test MAPE (%)	Test RMSE (\$)	Test R^2
Random Forest	23.50	14.49	22.18	278,645	0.9435
Gradient Boosting	42.95	42.53	44.28	368,130	0.9013
XGBoost	37.48	34.88	37.85	325,969	0.9226
LightGBM	38.20	36.50	38.98	338,576	0.9165

TABLE 1.1: Model comparison on the Class 5 cost estimation task. Values are taken directly from the organized modeling notebook (5-fold cross-validation and 80/20 train-test split).

Figures 1.8 and 1.9 visualize the same results. In Figure 1.8, a dashed horizontal line at 25% highlights the target Class 5 MAPE. Only the random forest model falls below this line on the held-out test set. Figure 1.9 shows that random forest also attains the highest R^2 score, indicating that it explains the largest share of variance in normalized costs.

1.5.2 Learning Curve and Generalization

To understand whether the chosen random forest model is prone to overfitting or underfitting, a learning curve is computed using 5-fold cross-validation across increasing training set sizes. Figure 1.10 plots the training and cross-validation MAPE as a function of the number of training examples.

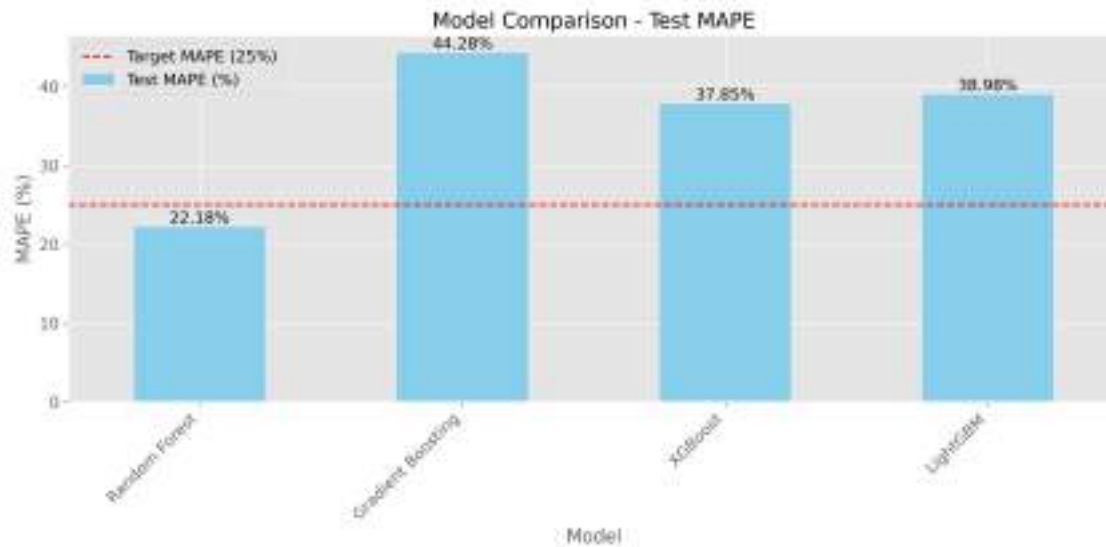


FIGURE 1.8: Comparison of test MAPE across models. The red dashed line indicates the 25% Class 5 performance target. Only the random forest model meets this target on the unseen test set.

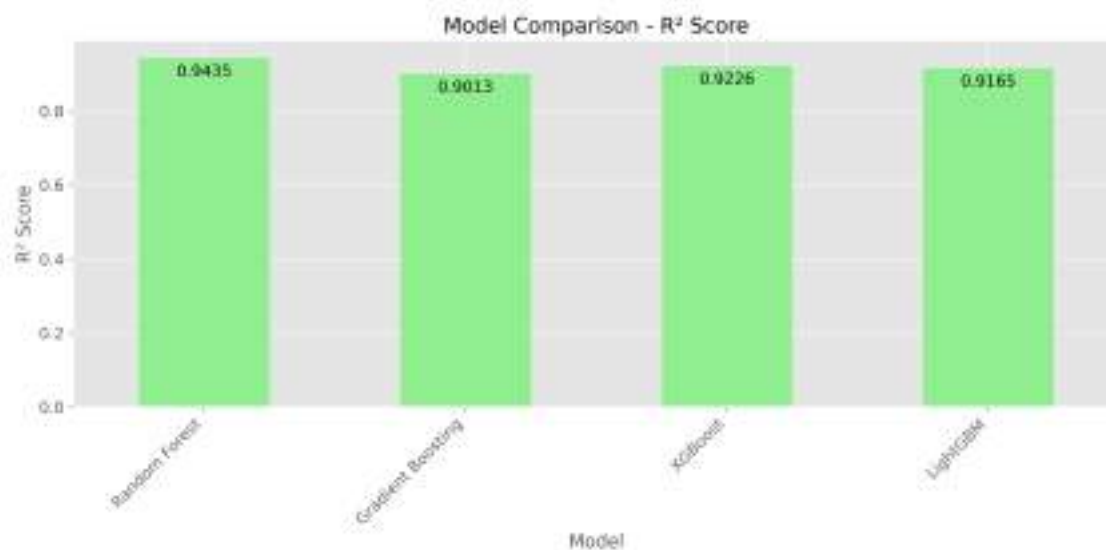


FIGURE 1.9: Comparison of test R^2 scores across models. Random forest explains the highest proportion of variance in normalized project costs.

At small training sizes, the model fits the training data very closely, and cross-validation errors are higher, which is typical. As the training set grows, both curves converge to the low 20% range. The gap between training and validation MAPE remains modest, suggesting that the model is not severely overfitting and that the

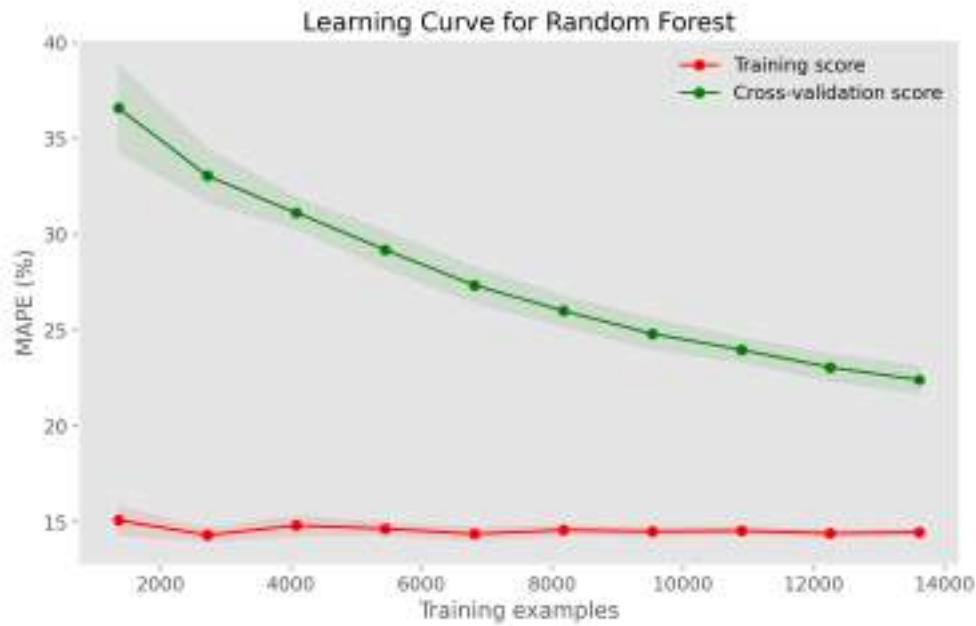


FIGURE 1.10: Learning curve for the random forest model. The narrowing gap between training and cross-validation MAPE suggests good generalization and that additional data may still yield incremental improvements.

selected level of complexity is appropriate for the available data. The curve also indicates that access to additional similar projects could further stabilize performance but is not strictly required to meet the current target.

1.5.3 Feature Importance and Driver Analysis

Tree-based models like random forests and gradient boosting provide feature importance scores that quantify how often and how effectively each feature is used to reduce prediction error across the ensemble. Figure 1.11 presents the top features from the random forest model.

The most important drivers include the official budget range, the CIQS complexity category (and its numeric encoding), project category (Water & Sewer vs. Civil vs. Site Work & Landscaping), and area cost factors. Location-related features such

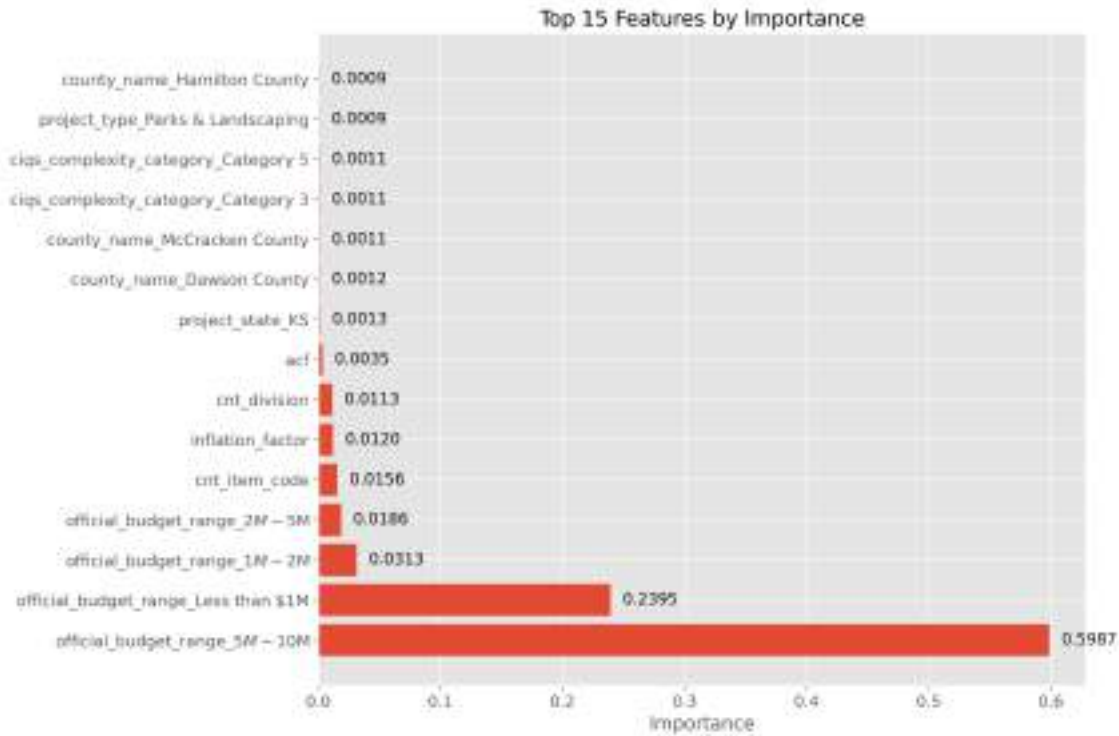


FIGURE 1.11: Top feature importances from the random forest model. Budget range, complexity, project category, and area cost factors are among the most influential predictors of normalized project cost.

as state, county, and region also contribute meaningfully, but no single geographic category dominates. This pattern is consistent with prior literature emphasizing the role of scope definition, complexity, and high-level functional type in early cost prediction [2–4].

1.5.4 Calibration and Residual Behavior

Calibration can be visualized by plotting actual versus predicted costs for the test set. Figures 1.12–1.15 show these scatter plots for all four models. Points lying near the 45-degree line correspond to well-predicted projects; systematic deviations from the line indicate bias.

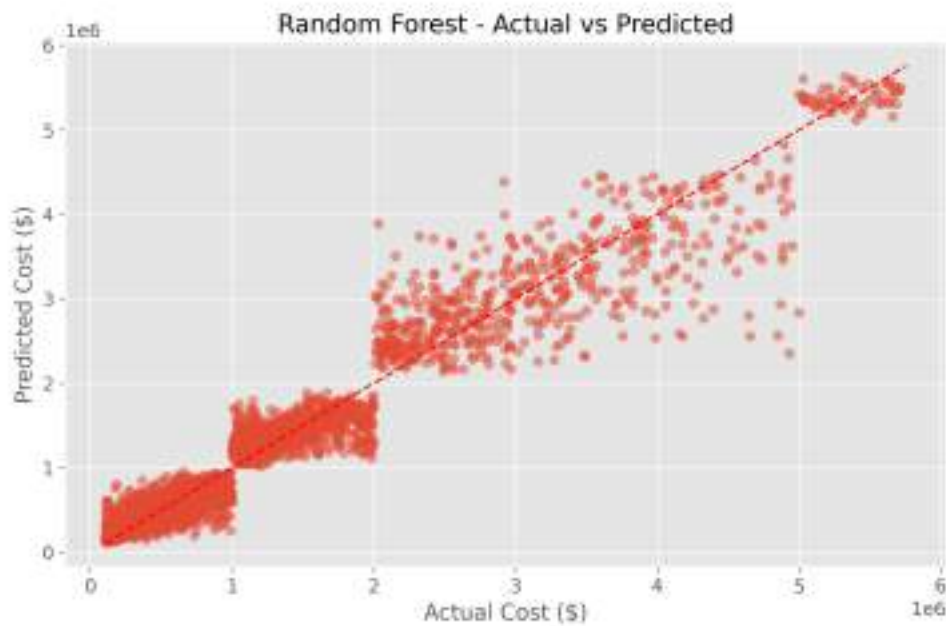


FIGURE 1.12: Actual versus predicted normalized costs for the random forest model on the test set. Points are reasonably concentrated around the 45-degree line, with some spread at the high-cost tail.

The random forest scatter plot shows the tightest concentration around the diagonal, especially for mid-range projects, which explains its superior MAPE and RMSE. For all models, predictions for the most expensive projects show greater dispersion, which is expected because fewer such projects exist in the training data and the underlying cost drivers may be more heterogeneous.

Residual analysis provides another lens. Figure 1.16 plots residuals (predicted minus actual) as a function of predicted cost for the random forest model. The residuals are roughly centered around zero, with greater spread at higher cost levels. There is no strong curvature or obvious pattern, which suggests that the model captures the major nonlinearities in the data. However, the slightly wider residual band at the high-cost tail indicates that additional features related to special design conditions, unique materials, or unusual site constraints might further reduce error for those projects.

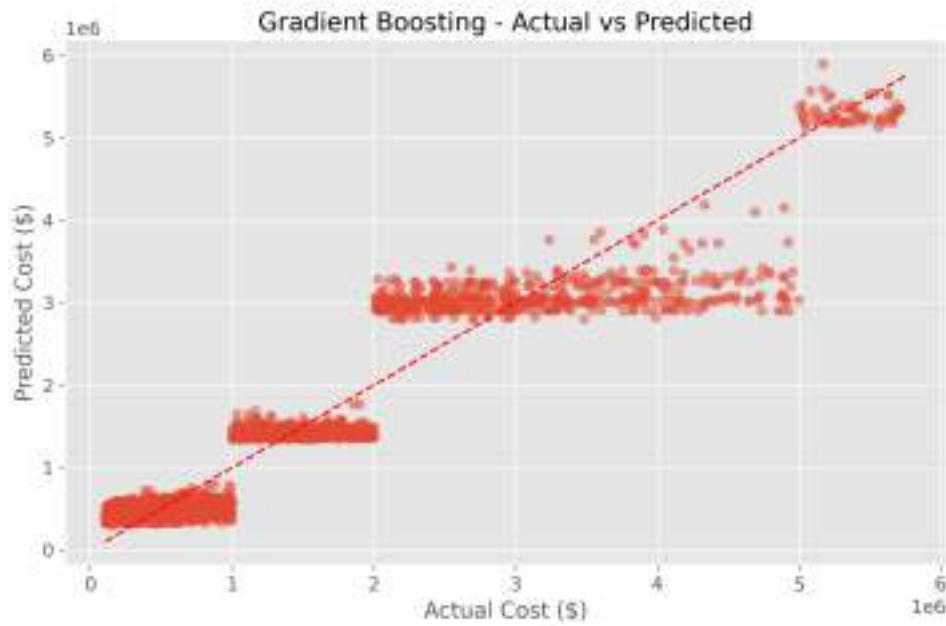


FIGURE 1.13: Actual versus predicted normalized costs for the gradient boosting model. The wider spread and underestimation of high-cost projects correspond to the higher MAPE and lower R^2 .

1.6 Discussion

The modeling results demonstrate that a relatively straightforward random forest model, built on a carefully engineered dataset and well-designed preprocessing pipeline, can achieve Class 5 cost estimation performance that meets or exceeds typical industry expectations. With a test MAPE of about 22.2%, the model falls within the accuracy ranges described by AACE for early estimates and surpasses the internal goal of 25% average error [1, 12]. The improvement is particularly notable given the diversity of project types and locations and the focus on relatively minimal, conceptual-level inputs.

The EDA shows that much of the explainable variation in cost can be captured by a combination of high-level scope descriptors (project category and type), engineered

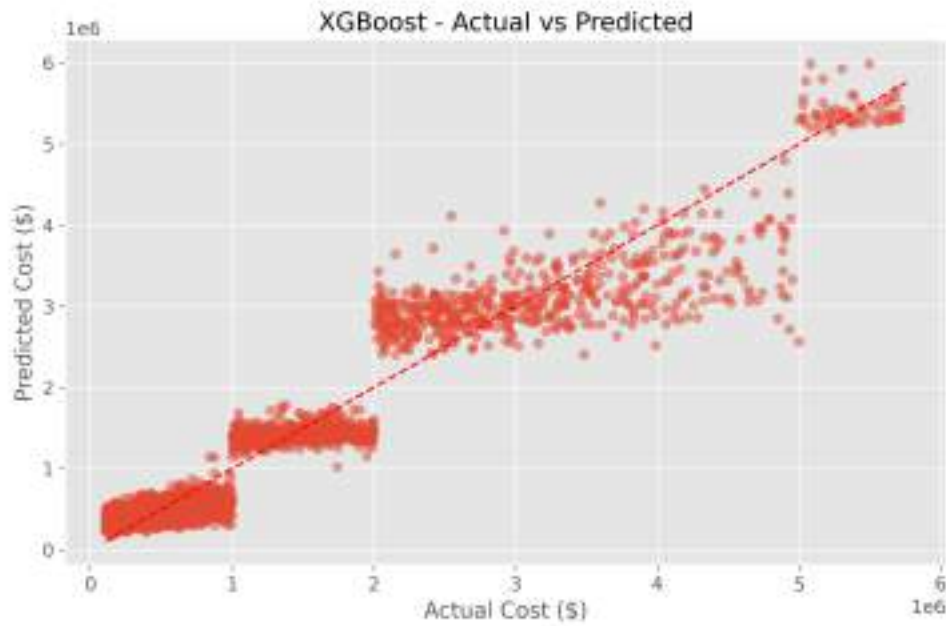


FIGURE 1.14: Actual versus predicted normalized costs for the XGBoost model. Predictions are better calibrated than gradient boosting but still lag the random forest in overall accuracy.

complexity metrics, budget range information, and regional context via area cost factors and clustered regions. This aligns with prior findings that emphasize the importance of combining project characteristics with environmental and economic indicators in early cost models [2–5, 16]. At the same time, the residual analysis indicates that very large or highly specialized projects remain more difficult to predict exactly, which is consistent with broader experience in the literature [9–11].

From a business perspective, the current model already serves as the engine behind a prototype Class 5 estimation tool implemented as a Flask web application. The app provides fast, data-driven cost estimates based on limited conceptual inputs and can be extended to surface confidence bands, scenario comparisons, and project archives. The engineered features and clean dataset are structured to support this deployment: the same preprocessing steps used in training are wrapped into a reusable prediction function that the Flask backend calls whenever users submit new project descriptors.

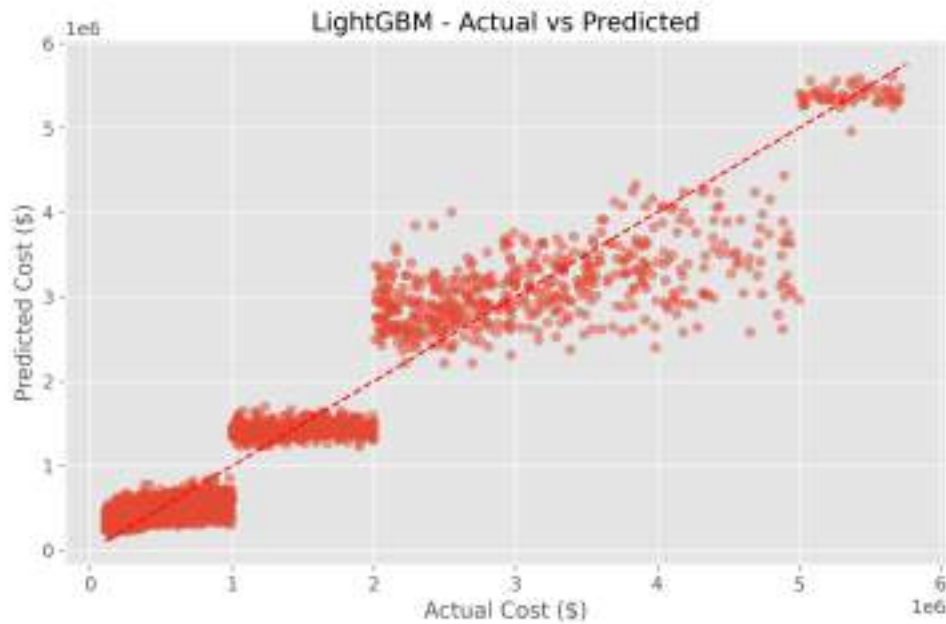


FIGURE 1.15: Actual versus predicted normalized costs for the LightGBM model. Performance is similar to XGBoost, with a slight underestimation tendency at the upper end of the cost range.

1.7 Limitations and Future Work

Despite the encouraging results, several limitations should be acknowledged. First, the modeling dataset covers only four TA-selected project types aggregated into three broader categories. While this focused scope is appropriate for a pilot tool and yields reliable performance, the model is not directly applicable to all PCS project types without retraining on additional data. Second, although CPI normalization addresses general inflation, localized market shocks, material price spikes, and supply chain disruptions are only partially captured through area cost factors and may require more explicit economic indicators for future models [16].

Third, the current models are point estimators: they provide a single best-guess cost, but they do not explicitly quantify uncertainty beyond what can be inferred from

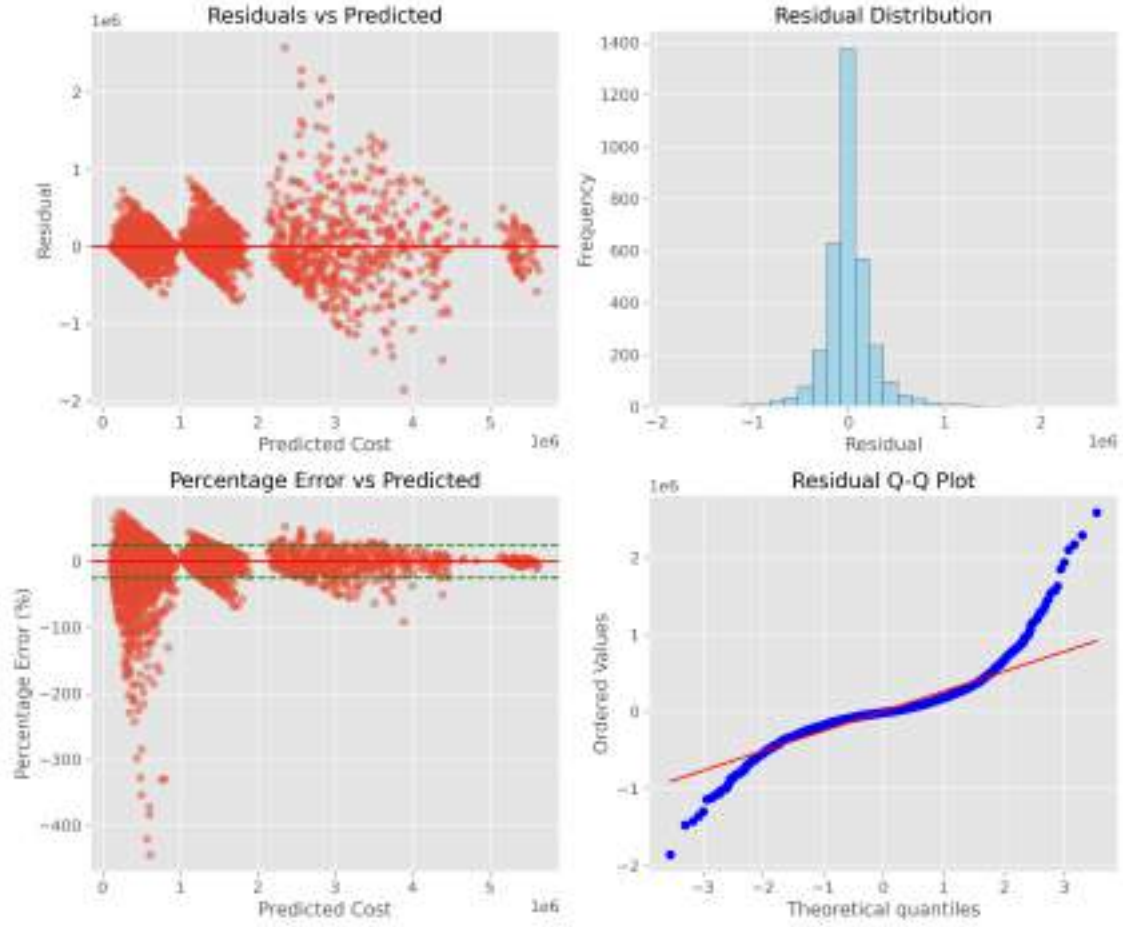


FIGURE 1.16: Residual analysis for the random forest model. Residuals are centered near zero with increasing variance at higher cost levels, reflecting the inherent difficulty of predicting very large projects.

historical MAPE. In practice, decision-makers often prefer to see cost ranges or probability distributions. Extending the model to provide prediction intervals, quantile estimates, or scenario-based outputs would align better with contingency and risk practices discussed in the cost estimation literature [16, 17]. Finally, hyperparameter tuning was intentionally kept modest in this phase to prioritize interpretability and speed. There is room to explore more sophisticated model classes and tuning strategies, such as gradient-boosted trees with optimized depth and learning rates, or explainable deep learning approaches where justified [5, 10].

Future work will focus on three directions. The first is further development and

hardening of the Flask-based web application delivered with this project, including enhanced input validation, richer reporting and visualization of estimates, user management, and improved logging and monitoring to support production use. The second is dataset expansion: incorporating additional project types, more granular schedule and scope descriptors, and richer economic indices, while maintaining the robust cleaning and normalization steps documented in the EDA notebooks. The third is uncertainty quantification: integrating probabilistic methods or ensemble techniques that explicitly model variance in cost outcomes, so that users can see not only a point estimate but also the likely range of costs for a proposed project.

Taken together, these steps will turn the present research prototype and its companion Flask application into a practical, extensible Class 5 estimation system that is grounded in both industry data and the broader insights from machine learning research in construction cost prediction.

1.8 Methodology

To further identify the underlying patterns and similarities across projects within the dataset, we applied a non-parametric approach to segment the dataset into sub-groups with more similar characteristics, and fitted ensemble methods on each sub-group. The goal of the approach is to explore the complex relationships among the variables that might potentially uncover patterns unique to sub-groups of projects, which could better explain the relationship between the predictors and the response variable.

Given the dataset used in the analysis and the findings from the exploratory data analysis, K-means clustering poses as a good candidate for an unsupervised non-parametric model since it is computationally efficient and more interpretable in

comparison to other clustering approaches. K-means clustering partitions the data into K clusters ($C_1, C_2, \dots, C_K$), represented by their means, also called centers. The center of each cluster is calculated as the mean of all instances belonging to that particular cluster using the formula below.

$$\mu_k = \frac{1}{N_k} \sum_{q=1}^{N_k} x_q \quad (1.1)$$

where N_k is the number of observations belonging to cluster k and μ_k is the mean of the cluster k .

To determine the optimal set of variables and the ideal number of clusters, we used an iterative approach where different groupings of variables and cluster sizes were used to fit the model and evaluated, ultimately selecting those that yielded the best performance. The variable set, complexity_numeric, cnt_division, and cluster size k=5 had the best performance and the highest overall ratio of the between-cluster sum of squares and the total sum of squares of 95.14%. Fig X1 shows the clusters identified by the model.

The observations in the dataset were assigned to their respective clusters as predicted by the K-means clustering model, segmenting the dataset into five subsets of data. The number of observations in each subgroup are as follows: ($C_1 = 3553, C_2 = 3250, C_3 = 4857, C_4 = 1887$, and $C_5 = 3478$). Each subset of data was further divided into training and testing datasets, using a 80/20 split, after which ensemble methods, including Decision Tree and Random Forest, were applied to each subgroup.

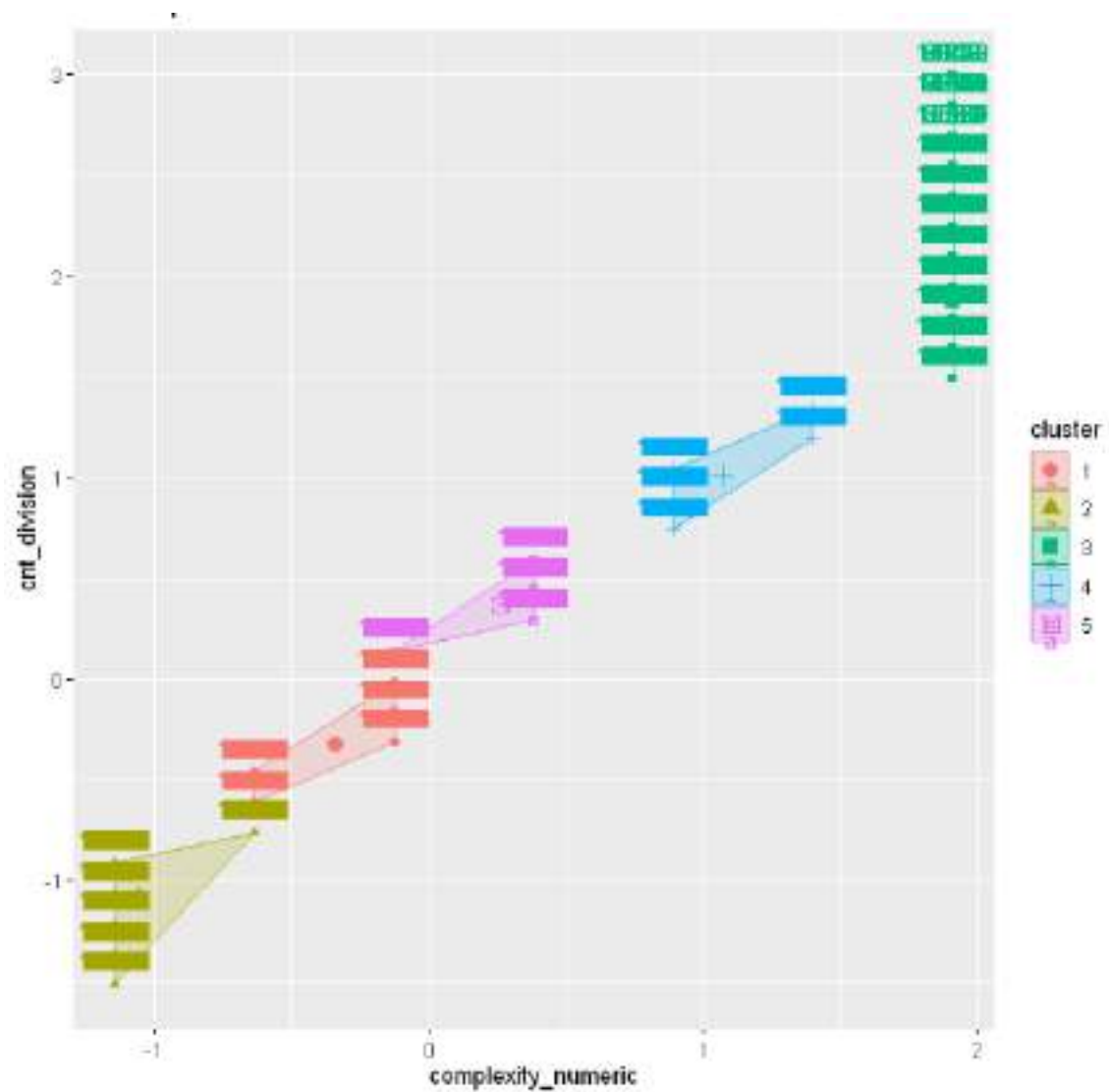


FIGURE 1.17: Cluster assignment using count of divisions and project complexity variables.

1.9 Results

The parameters used to fine-tune the models were determined using cross-validation method, and the following metrics were used to evaluate the performance of the models: R-squared, and Mean Absolute Error and Mean Absolute Percentage Error as per the recommendation of the project sponsor. The results of the metrics based

on cross-validation for each of the data subsets are presented on Table X1 and Table X2.

Cluster	R^2 (%)	MAE (%)	MAPE (%)
C_1	90.22	195731.49	27.70
C_2	87.63	234015.15	18.48
C_3	91.37	165937.66	20.99
C_4	86.70	308450.63	15.11
C_5	86.28	169533.77	34.71

TABLE 1.2: Performance Metrics of Decision Tree Model across 5 data subgroups identified using K-means clustering

Cluster	R^2 (%)	MAE (%)	MAPE (%)
C_1	92.34	182151.97	27.20
C_2	90.80	218662.05	18.68
C_3	92.57	163062.74	22.28
C_4	90.19	287794.80	14.51
C_5	88.74	160894.62	34.04

TABLE 1.3: Performance Metrics of Random Forest Model across 5 data subgroups identified using K-means clustering

The results displayed on the tables above, show variations in the performance of both the Decision Tree model and the Random Forest model across the different data subgroups, with the data subgroups C_4 and C_5 having the lowest and highest MAPE, respectively, as evaluated for both models.